



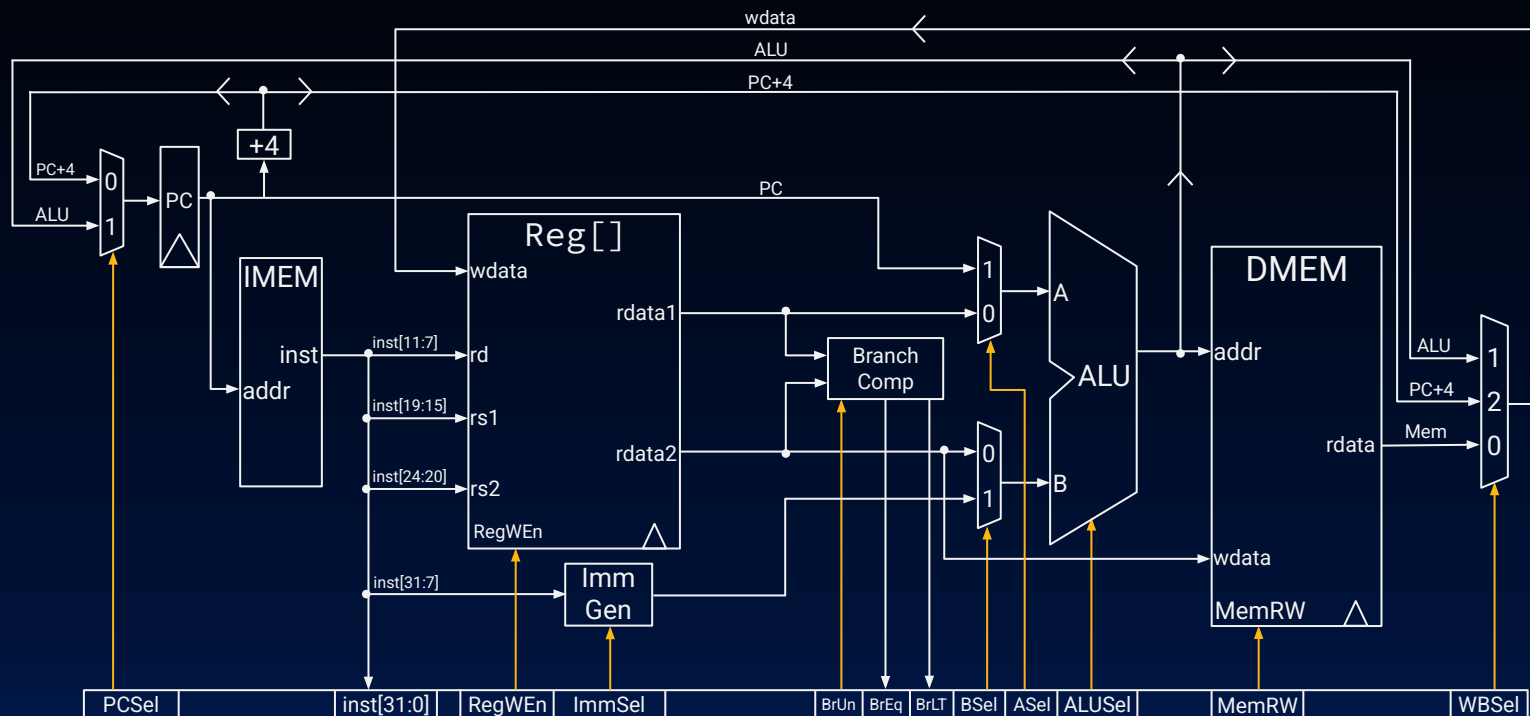
CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

RISC-V Control Logic, Instruction Timing



This datapath is capable of executing every RV32I instruction, one cycle per instruction.

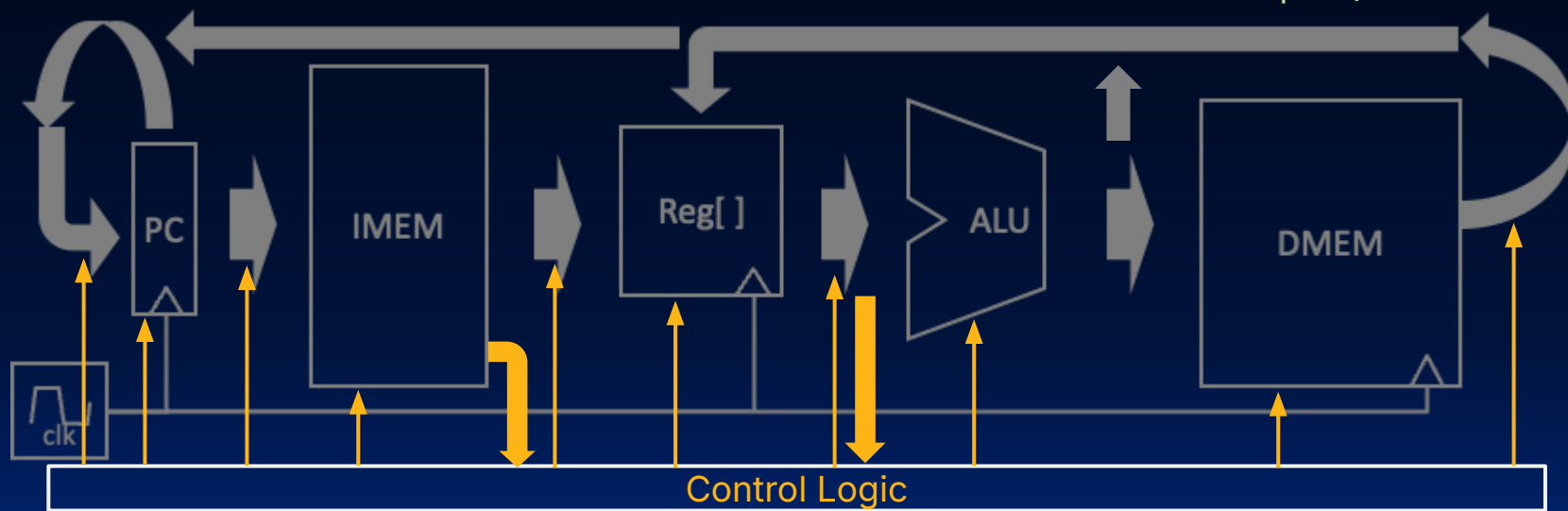
- The control logic selects “needed” datapath lines based on the instruction.
 - The controller therefore specifies how to execute instructions.



Space Station
(Datapath)



Mission Control
(Control Logic)





[Practice]

See reference card on [course website](#).

For each instruction below:

- Which wires should hold relevant data, i.e., what is the “lit up” datapath?
- What should the control signals be set to?
- Does this instruction use all 5 phases?
- What is this instruction’s **critical path**?

1. `beq rs1 rs2 offset`

Critical path: The path between input(s) and output(s) that incurs worst-case/max delay.

- Used to determine clock period
- Compute: Longest delay **from a clocked state output to another clocked state input**.



Yan, SP26





Control Logic Design: ROM

- Control Logic Design: ROM
- Instruction Timing
- Performance Measures
- [Extra] Control Logic Design: Combinational Logic

Control Logic Truth Table (Partial)

Do Project 3!

inst[31:0]	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
R-Type	*	*	+4	*	*	Reg	Reg	(Op)	Read	1	ALU
add								Add			
sub								Sub			
addi	*	*	+4	I	*	Reg	Imm	Add	Read	1	ALU
lw	*	*	+4	I	*	Reg	Imm	Add	Read	1	Mem
sw	*	*	+4	S	*	Reg	Imm	Add	Write	0	*
beq				B	*	PC	Imm	Add	Read	0	*
not taken	0	*	+4								
taken	1	*	ALU								
bne				B	*	PC	Imm	Add	Read	0	*
taken	0	*	ALU								
not taken	1	*	+4								
blt taken	*	1	ALU	B	0	PC	Imm	Add	Read	0	*
bltu taken	*	1	ALU	B	1	PC	Imm	Add	Read	0	*
jalr	*	*	ALU	I	*	Reg	Imm	Add	Read	1	PC+4
jal	*	*	ALU	J	*	PC	Imm	Add	Read	1	PC+4
auipc	*	*	+4	U	*	PC	Imm	Add	Read	1	ALU

Yan, SP26

Two Options for Control Realization

- Read-Only-Memory (ROM)
 - Regular structure
 - Can be easily reprogrammed to...
 - fix errors
 - add new instructions
- Combinational Logic
 - Instead of ROM, design with gates (AND, OR, NOT)
 - Popular for chip design using logic synthesis tools (fast)

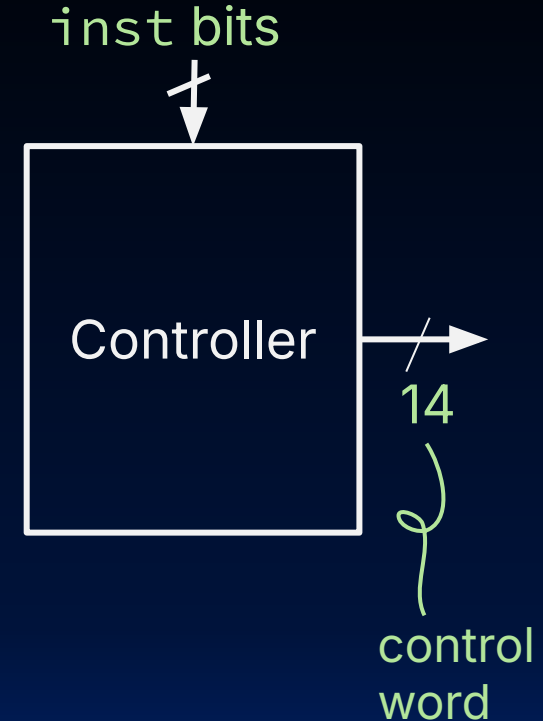
Project 3 suggestion:
Design control logic with
ROM.





Two Options for Control Realization

- Read-Only-Memory (ROM)
 - Regular structure
 - Can be easily reprogrammed to...
 - fix errors
 - add new instructions
- Combinational Logic
 - Instead of ROM, design with gates (AND, OR, NOT)
 - Popular for chip design using logic synthesis tools (fast)





RV32I ROM-based Control

- **Read-Only-Memory** (ROM) reads out a word at a given "address".
- To get control signals:
 - Input Address: Relevant set of **instruction bits**
 - Output word: Bits of control signals

1. What is the minimum set of instruction bits needed for ROM input?

2. Why might PCSel not be known at this time?

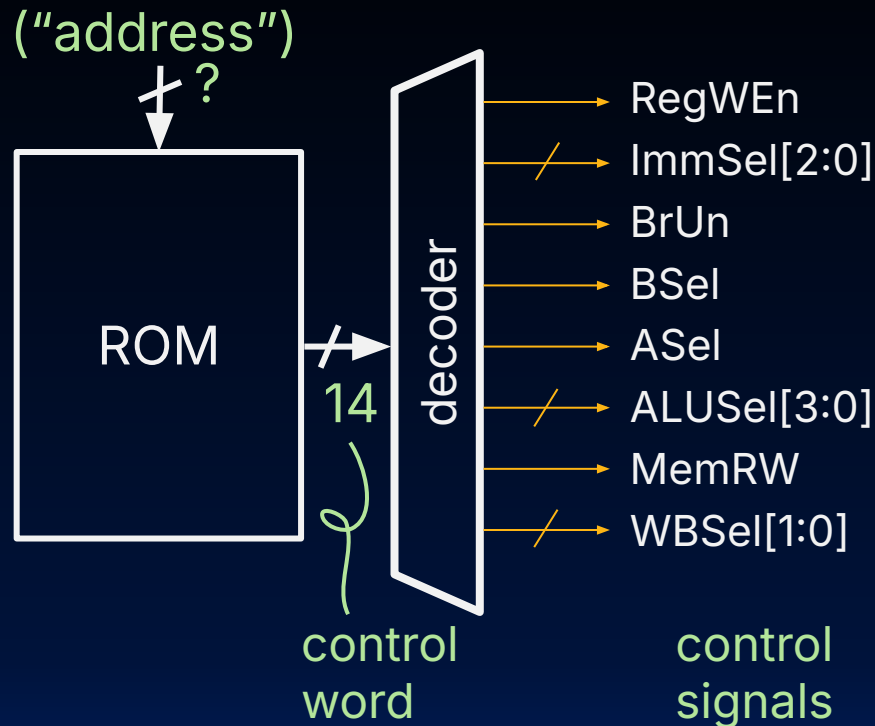
A. 32

B. 17

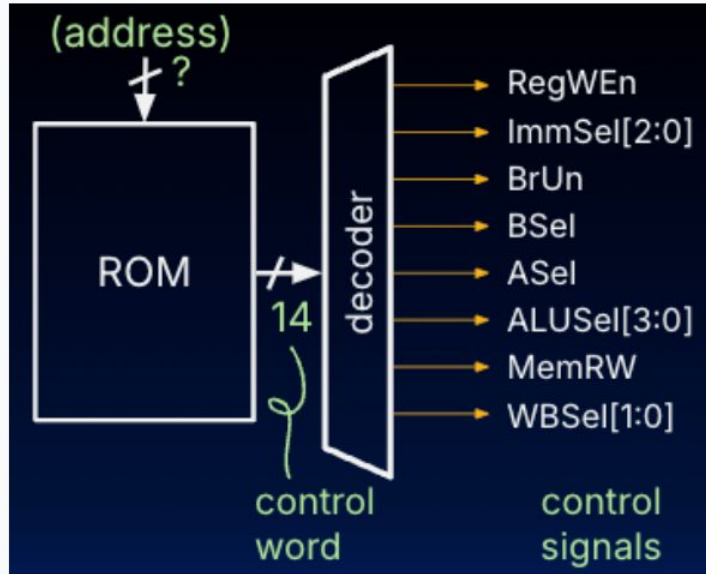
C. 11

D. 9

E. Something else



L25 What is the minimum set of instruction bits needed for input? Assume RV32I.



A. 32

0%

B. 17

0%

C. 11

0%

D. 9

0%

E. Something else

0%



1. RV32I is a “9-bit ISA”

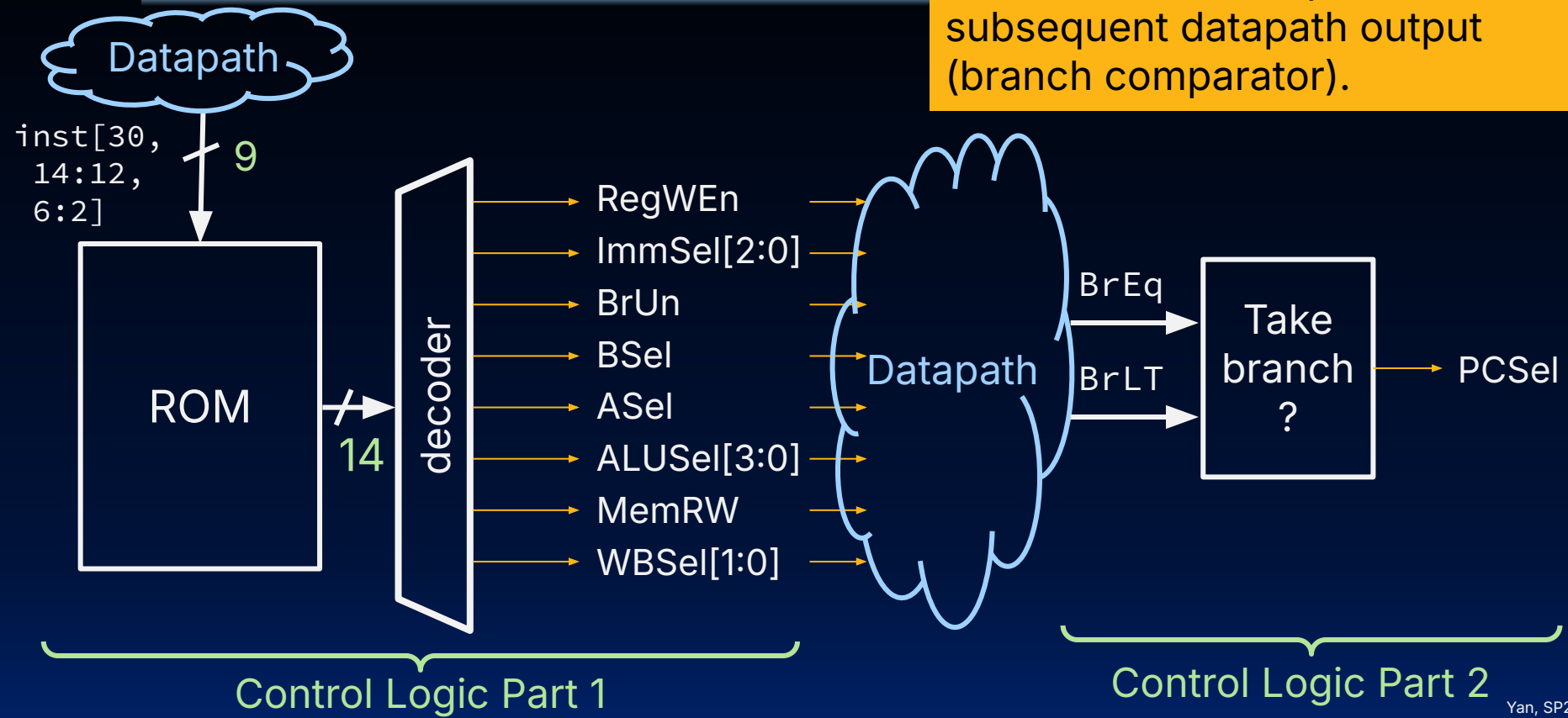
- `inst[30]`
 - `funct7`
 - `add/sub, sll/srl`
- `inst[14:12]`
 - `funct3`
- `inst[6:2]`
 - `opcode`
 - `inst[1:0]` fixed in RV32I, changes with extensions (e.g., compressed instructions)

In RV32I, the instruction type is encoded using only **9 bits**.

imm[31:12]				rd	0110111	LUI	
imm[31:12]				rd	0010111	AUIPC	
imm[20:10:1 11 19:12]				rd	1101111	JAL	
imm[11:0]		rs1	000	rd	1100111	JALR	
imm[12:10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ	
imm[12:10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE	
imm[12:10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT	
imm[12:10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE	
imm[12:10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU	
imm[12:10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU	
imm[11:0]		rs1	000	rd	0000011	LB	
imm[11:0]		rs1	001	rd	0000011	LH	
imm[11:0]		rs1	010	rd	0000011	LW	
imm[11:0]		rs1	100	rd	0000011	LBU	
imm[11:0]		rs1	101	rd	0000011	LHU	
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB	
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH	
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW	
imm[11:0]		rs1	000	rd	0010011	ADDI	
imm[11:0]		rs1	010	rd	0010011	SLTI	
imm[11:0]		rs1	011	rd	0010011	SLTIU	
imm[11:0]		rs1	100	rd	0010011	XORI	
imm[11:0]		rs1	110	rd	0010011	ORI	
imm[11:0]		rs1	111	rd	0010011	ANDI	
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
fm	pred	succ	rs1	000	rd	0001111	FENCE
0000000000000			00000	000	00000	1110011	ECALL
0000000000001			00000	000	00000	1110011	EBREAK

2. RV32I Control: PCSel

PCSel cannot be encoded in our ROM because it depends on subsequent datapath output (branch comparator).





Instruction Timing

- Control Logic Design: ROM
- Instruction Timing
- Performance Measures
- [Extra] Control Logic Design: Combinational Logic



Instructions' Critical Path

See reference card on [course website](#).

For each of the below instructions, what is the delay along the **critical path**?

1. `add rd rs1 rs2`
2. `lw rd imm(rs1)`

- A. $t_{clk_q} + t_{Add} + t_{IMEM} + t_{RegFile} + t_{BrComp} + t_{ALU} + t_{DMEM} + t_{mux} + t_{Setup}$
- B. $t_{clk_q} + t_{IMEM} + t_{RegFile} + t_{mux} + t_{ALU} + t_{mux} + t_{Setup}$
- C. $t_{clk_q} + t_{IMEM} + \max\{t_{RegFile}, t_{Imm}\} + t_{ALU} + 2 * t_{mux} + t_{DMEM} + t_{Setup}$
- D. $t_{clk_q} + t_{IMEM} + \max\{t_{RegFile}, t_{Imm}\} + t_{ALU} + 3 * t_{mux} + t_{Setup}$
- E. Something else



Yan, SP26





Instructions: Critical Path (2/2)

`add rd rs1 rs2`

Critical path delay = **// B**

$$\begin{aligned} & \text{tclk_q} + \max \{ \text{tAdd} + \text{tmux}, \\ & \hspace{15em} \text{tIMEM} + \text{tRegFile} + \text{tmux} + \text{tALU} + \text{tmux} \} + \text{tSetup} \\ & = \text{tclk_q} + \text{tIMEM} + \text{tRegFile} + \text{tmux} + \text{tALU} + \text{tmux} + \text{tSetup} \end{aligned}$$

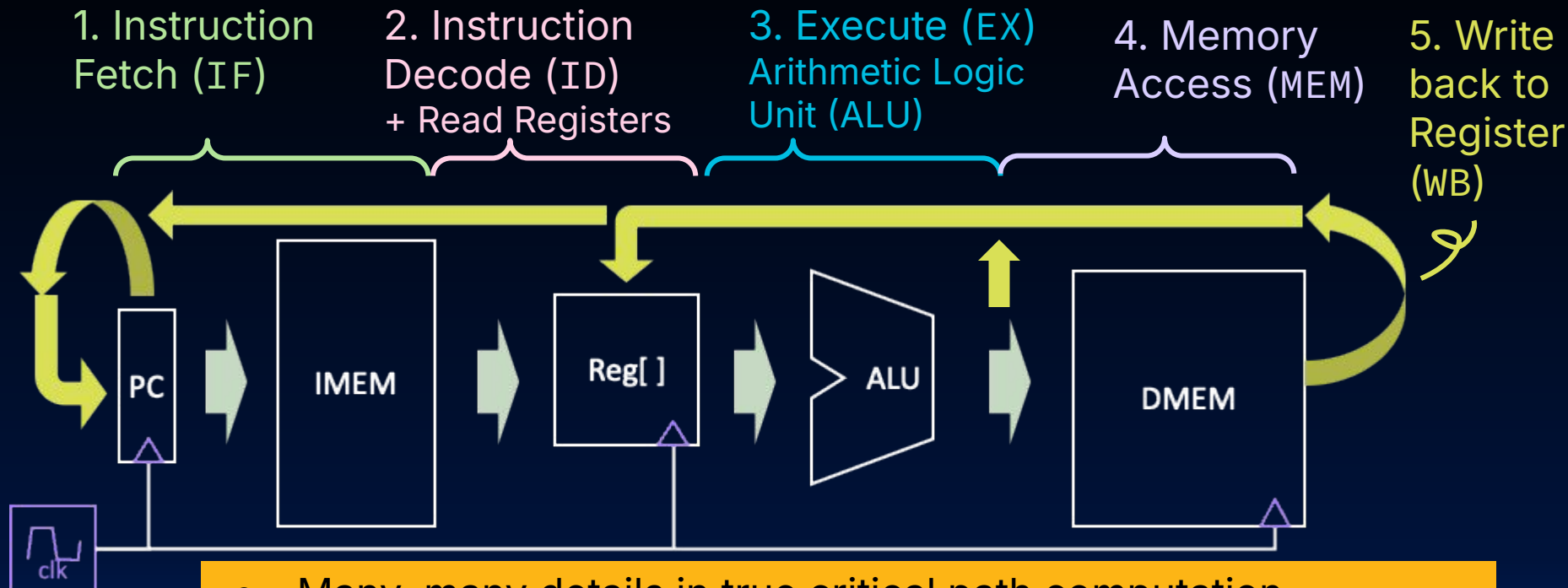
Not all HW units used by all instructions. The **critical path changes** based on instruction.

`lw rd imm(rs1)`

Critical path delay = **// C**

$$\begin{aligned} & \text{tclk_q} + \max \{ \text{tAdd} + \text{tmux}, \\ & \hspace{15em} \text{tIMEM} + \text{tImm} + \text{tmux} + \text{tALU} + \text{tDMEM} + \text{tmux}, \\ & \hspace{15em} \text{tIMEM} + \text{tRegFile} + \text{tmux} + \text{tALU} + \text{tDMEM} + \text{tmux} \} \\ & \quad + \text{tSetup} \\ & = \text{tclk_q} + \text{tIMEM} + \max\{\text{tRegFile}, \text{tImm}\} \\ & \quad + \text{tALU} + 2 * \text{tmux} + \text{tDMEM} + \text{tSetup} \end{aligned}$$

How to set clock frequency? (1/2)

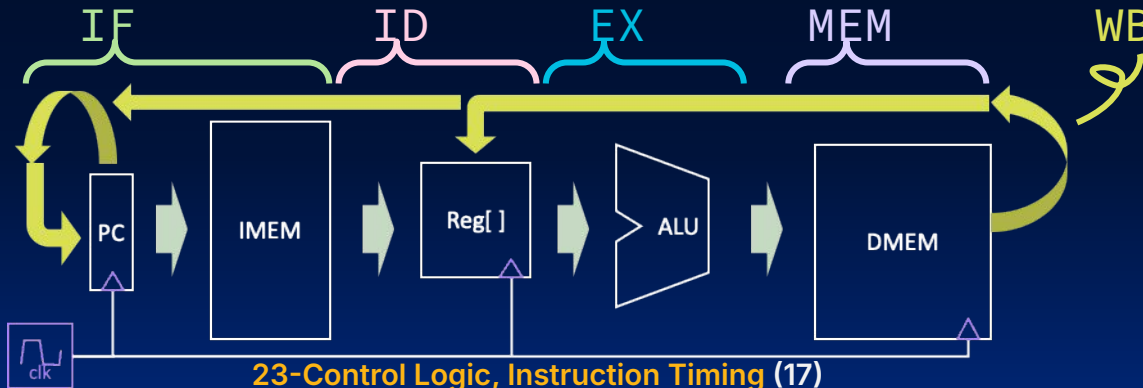


- Many, many details in true critical path computation...
- To estimate maximum clock frequency, analyze at a higher level and assume **phases are dominated by major HW units.**

For each instruction, which phases are “active”?

- Not all instructions use all HW.
- Not all instructions are active in all five phases!

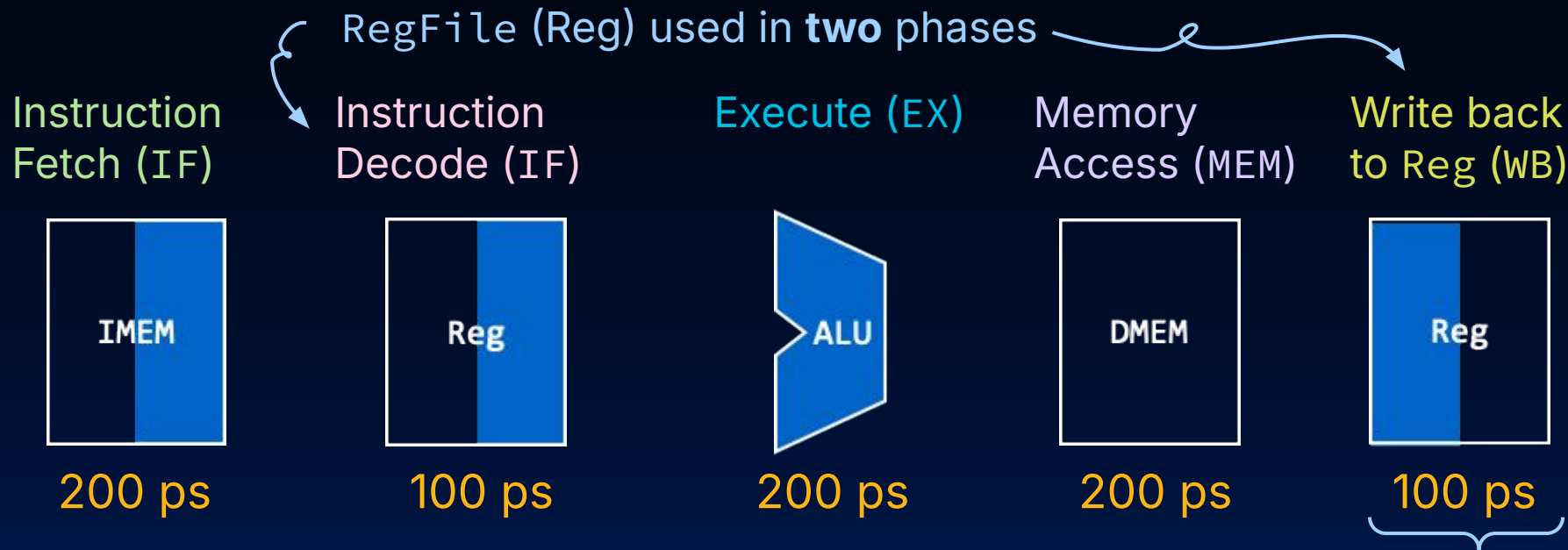
Instruction	Instruction Fetch (IF)	Instruction Decode (ID)	Execute ALU (EX)	Memory Access (MEM)	Write Back (WB)
add	X	X	X		X
beq					
jal					
lw	X	X	X	X	X
sw					





How to set clock frequency? (2/2)

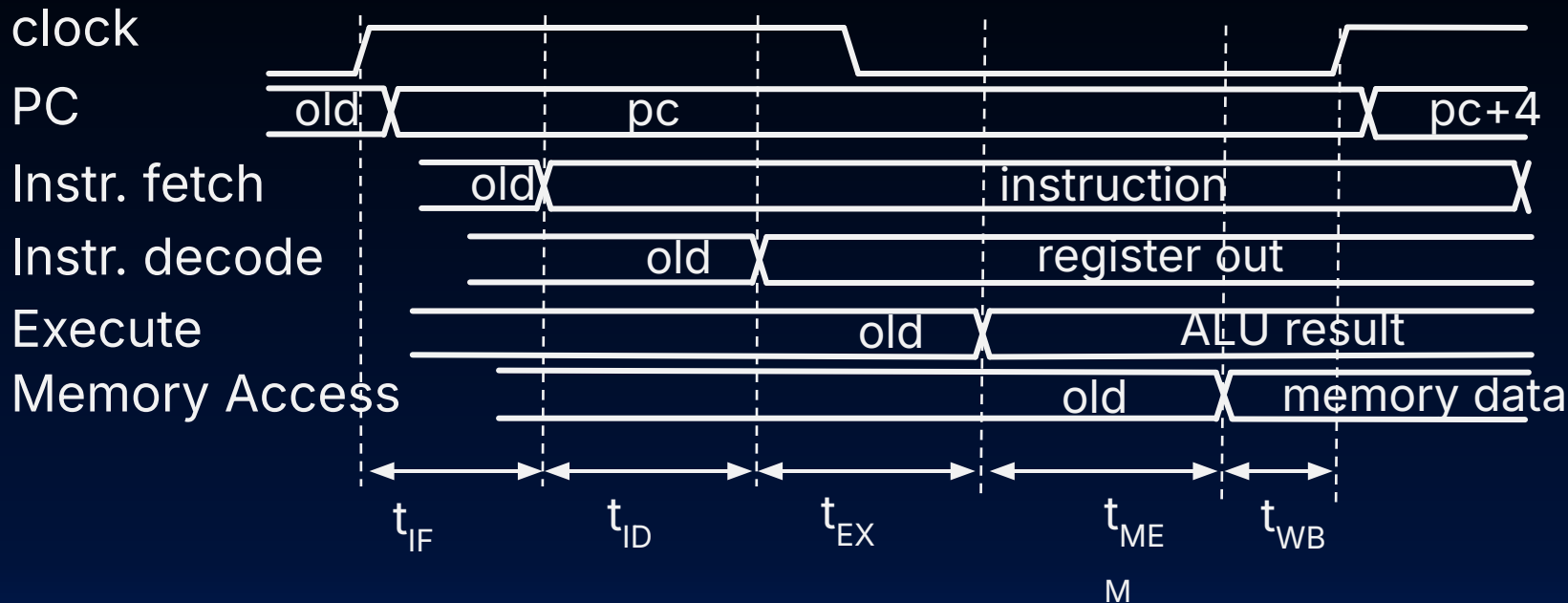
Assume each phase is **dominated** by major functional HW units:





5-Phase Timing, Waveforms

[reference]





Compute Clock Period

[Got to here](#)

- The clock cycle must encompass the **longest critical path delay** incurred by any instruction.

Instruction	IF (200ps)	ID (100ps)	EX (200ps)	MEM (200ps)	WB (100ps)	Total
add	X	X	X		X	600ps
beq	X	X	X			500ps
jal	X	X	X			500ps
lw	X	X	X	X	X	800ps
sw	X	X	X	X		700ps

- Maximum Clock Frequency:
 - $f_{\max} = 1/800 \text{ ps} = 1.25 \text{ GHz}$
 - However, not all instructions use all HW blocks.
- Many blocks are idle most of the time...!
 - Conversely, each phase just takes 200 ps → could clock 5GHz...??

Yan, SP26

Performance Measures

- “Our” RISC-V Single-Cycle Datapath executes instructions at 1.25GHz.
 - 1 instruction every 800 ps
 - Frequency unit $\text{Hz} = \text{s}^{-1}$

$$\frac{1}{800} \frac{\text{instr}}{\text{ps}} \cdot 10^{12} \frac{\text{ps}}{\text{s}} \cdot 10^{-9} \frac{\text{GHz}}{\text{Hz}} = 1.25 \text{ GHz}$$

- Can we improve our processor's performance?
- Not so obvious what we mean...
 - Instruction execution time?
 - Total # instructions executed per unit time?
 - Energy per instruction?
 - Cost over lifetime of chip?

} This
course

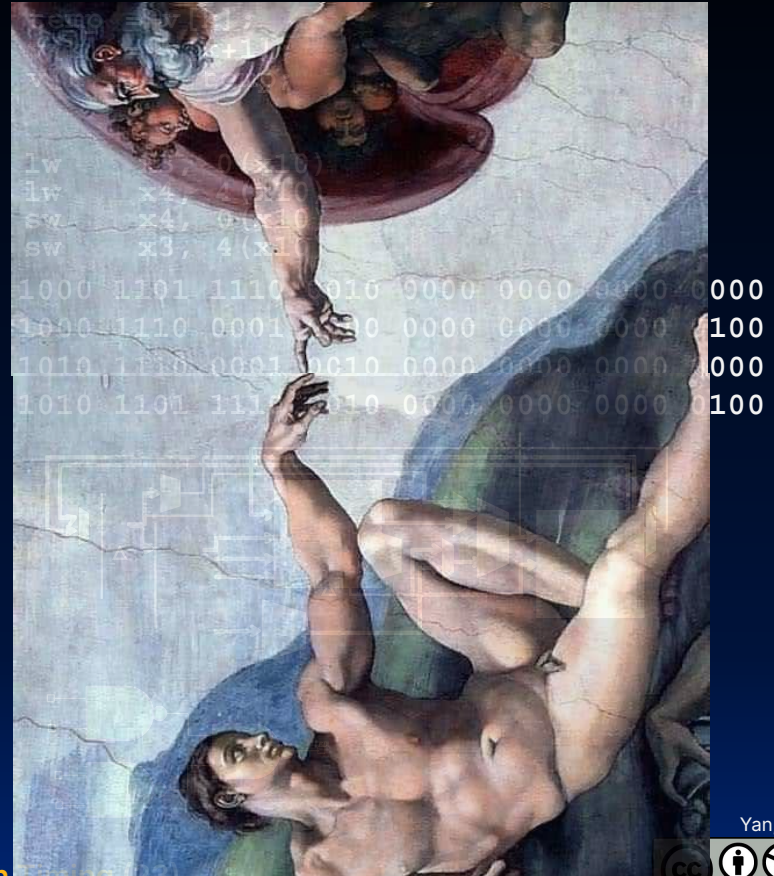
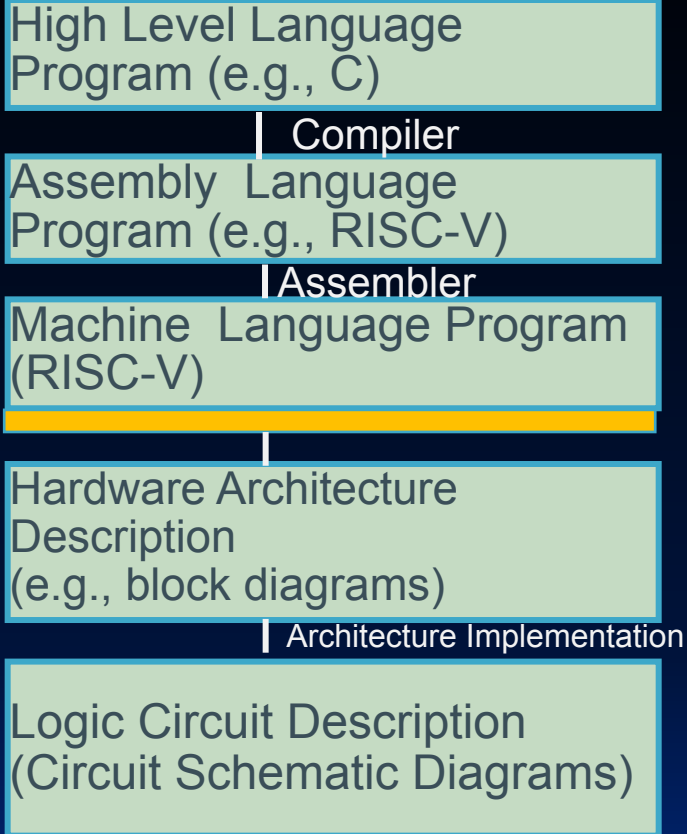


Agenda

**"And In
Conclusion..."
(drum roll)**

- Control Logic Design: ROM
- Instruction Timing
- Performance Measures
- [Extra] Control Logic Design: Combinational Logic

Call home, we've made HW/SW contact!



"And in conclusion..."

- We have built a processor!
 - Capable of executing all RISC-V instructions in one cycle each
- Not all HW units used by all instructions
 - The **critical path changes** based on instruction.
- 5 Phases of execution
 - IF, ID, EX, MEM, WB
 - Not all instructions are active in all phases
- Controller specifies how to execute instructions
 - Implemented as ROM or logic



Performance Measures

- Control Logic Design: ROM
- Instruction Timing
- Performance Measures
- [Extra] Control Logic Design: Combinational Logic

Great Idea #5: Performance Measurement & Improvement

Match application to underlying hardware to exploit:

[Lec 01]

- Locality;
- Parallelism;
- Special hardware features, like specialized instructions (e.g., matrix manipulation).

Latency/Throughput:

- **How long** to set the problem up and complete it (or how many tasks can be completed in a given time)
- How much faster does it execute **once it gets going**

} today

Transportation Analogy

	Sports Car	Bus
		
Passenger Capacity	2 people	50 people
Travel Speed	200 mph	50 mph
Gas Mileage	5 mpg	2 mpg

Task: A 50-mile round trip for 100 passengers.

Which car performs **"better"**?

- Depends on the **performance measure!**

Trip time per ride	15 minutes ✓	60 min
Time for 100 passengers	750 min (50 2-person trips)	120 min ✓ (2 50-person trips)
Gallons per passenger	5 gallons	0.5 gallons ✓

(also consider: cost over car lifetime...)

Measuring Performance in Computers

- Many different measures of computer performance:

Transportation Analogy	Measure	In a Computer
Trip time per ride	Latency	Instruction latency
Time for 100 passengers	Latency	Program execution time (e.g., time to update display)
Passengers/hour	Throughput	Total tasks per unit time (e.g., # of server requests handled per hour)
Gallons per passenger (not covered)	Energy Efficiency	Energy per task (e.g., how many movies you can watch per battery charge)

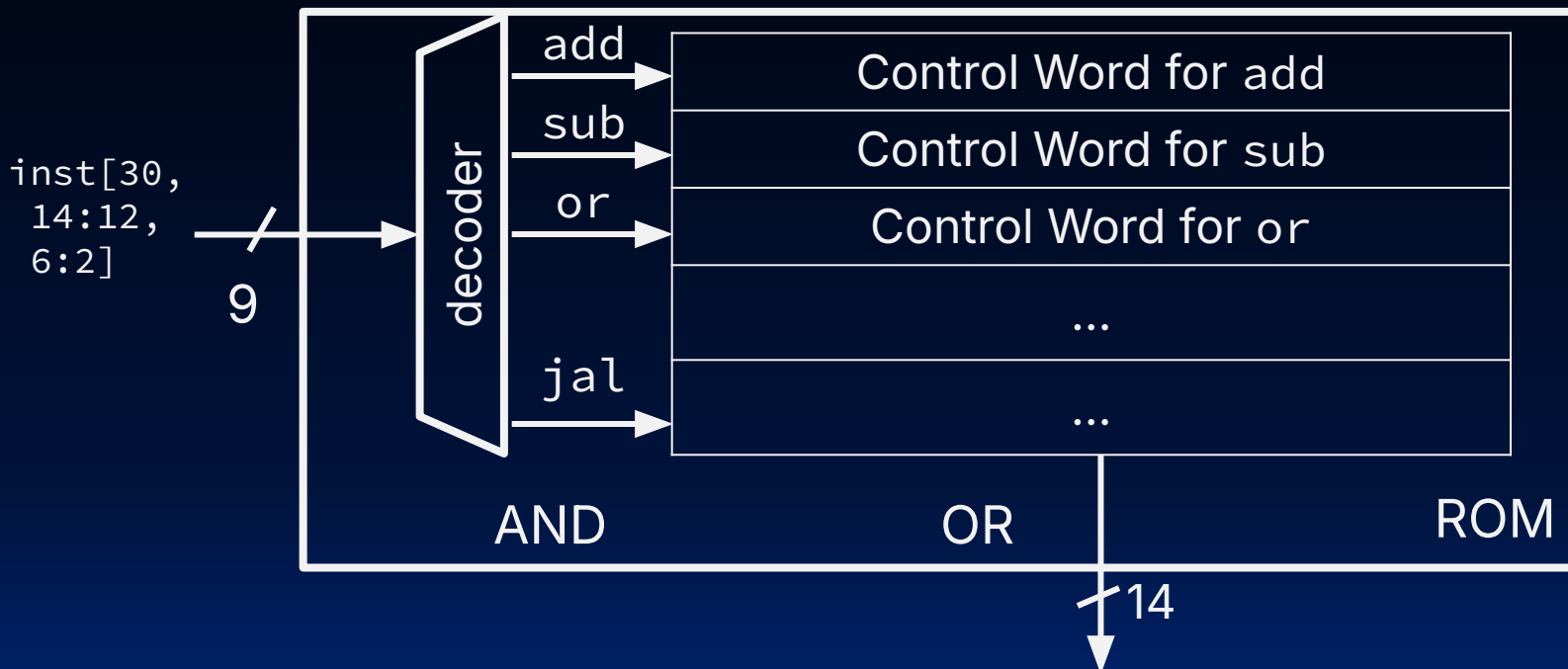


[Extra] Control Logic Design: Combinational Logic

- Control Logic Design: ROM
- Instruction Timing
- Performance Measures
- [Extra] Control Logic Design: Combinational Logic

ROM Controller Implementation

Under the hood, the ROM controller can be implemented as combinational logic gates. Use Sum of Products on logic table.





Combinational Logic Control: BrUn

Instruction	Opcode	Funct3
beq rs1 rs2 label	110 0011	000
bne rs1 rs2 label	110 0011	001
blt rs1 rs2 label	110 0011	100
bltu rs1 rs2 label	<u>110 0011</u>	<u>110</u>
bge rs1 rs2 label	110 0011	101
bgeu rs1 rs2 label	<u>110 0011</u>	<u>111</u>

31	25	24	20	19	15	14	12	11	7	6	0
B	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode					



Note: inst[1:0] fixed in RV32I, changes with extensions (e.g., compressed instructions)

BrUn = (B-Type) · (Unsigned Compare)

$$= (\text{inst}[6] \cdot \text{inst}[5] \cdot \overline{\text{inst}[4]} \cdot \overline{\text{inst}[3]} \cdot \overline{\text{inst}[2]}) \cdot (\text{inst}[13])$$

Yan, SP26



Combinational Logic Control: add

Instruction				Opcode		Funct3	Funct7	
add	rd	rs1	rs2	011	0011	<u>000</u>	<u>000</u>	0000
sub	rd	rs1	rs2	011	0011	000	010	0000



Note: $inst[1:0]$ fixed in RV32I, changes with extensions (e.g., compressed instructions)

31	25	24	20	19	15	14	12	11	7	6	0
R	funct7		rs2		rs1		funct3		rd		opcode

$$\begin{aligned} \text{add} &= (\text{inst}[30, 14:12] \text{ for add}) \cdot (\text{R-Type}) \\ &= \overline{\text{inst}[30]} \cdot \overline{\text{inst}[14]} \cdot \overline{\text{inst}[13]} \cdot \overline{\text{inst}[12]} (\text{R-Type}) \end{aligned}$$

$$\text{R-Type} = (\overline{\text{inst}[6]} \cdot \text{inst}[5] \cdot \text{inst}[4] \cdot \overline{\text{inst}[3]} \cdot \overline{\text{inst}[2]})$$